

Deploying a Docker App on Kubernetes (EKS)

[Terraform](#) · [Terragrunt](#) · [Kubernetes](#) · [Kustomize](#) · [Helm](#) · [ArgoCD](#)

Step-by-step beginner guide — Region: ap-south-1 (Mumbai) — EKS Auto Mode

Mubin Shaikh · October 2026

Beginner guide: the same app on Kubernetes (EKS) with Terraform, Terragrunt, Helm, Kustomize and ArgoCD

This is the Kubernetes version of `../docker-ec2-deploy`. Same Node app, same Docker image. The EC2 + ASG + CodeDeploy servers are replaced by an EKS cluster, and Jenkins/CodeDeploy are replaced by **GitOps**: ArgoCD watches this Git repo and applies whatever is in it.

Region: **ap-south-1 (Mumbai)**. Replace `664418953285` with your account ID everywhere you see it.

What you are building:

```
Terragrunt -> Terraform -> VPC + EKS (Auto Mode) + ECR + ArgoCD          (infra, run once)

docker build/push -> ECR
git push (new image tag) -> GitHub -> ArgoCD -> Kubernetes Deployments -> NLB -> users
```

Tool	What it does here	Folder
Terraform	Describes the AWS resources (VPC, EKS, ECR, ArgoCD install)	<code>terraform/modules/</code>
Terragrunt	Runs those modules per environment, keeps state in S3, passes outputs between them (VPC -> EKS -> ArgoCD)	<code>terragrunt/</code>
Kubernetes	Plain manifests for the app: a Deployment (pods) and a Service (load balancer)	<code>kustomize/base/</code>
Kustomize	Takes the plain manifests and changes a few things per environment (namespace, replicas, image tag)	<code>kustomize/overlays/</code>
Helm	Another way to package the same app: templates + <code>values.yaml</code>	<code>helm/sample-app/</code>
ArgoCD	Runs inside the cluster, pulls this repo and keeps the cluster matching it	<code>argocd/apps/</code>

Kustomize and Helm do the same job (one app, many environments). Both are here so you can compare them. In a real project you would pick one.

Cost warning

This creates real, billed resources. Rough monthly cost in ap-south-1 if you leave it running: EKS control plane ~\$73, NAT gateway ~\$35, 3 NLBs (one per app) ~\$50, Auto Mode nodes

~\$30-60. **About \$200/month.** Run the Cleanup phase as soon as you are done.

Phase 0 - Install the tools (your laptop or a Linux box)

You need: `aws` CLI v2, `terraform` `>= 1.10`, `terragrunt` `>= 1.0`, `kubectl`, `helm` v4, `kustomize` v5, `docker`, `git`.

```
aws sts get-caller-identity          # must show your account
terraform version; terragrunt --version; kubectl version --client; helm version; kustomize v
ersion
```

Phase 1 - Look at the layout

k8s-eks-deploy/	
terraform/modules/network	VPC: 2 public + 2 private subnets, 1 NAT gateway, subnet tags for load balancers
terraform/modules/eks	EKS cluster in Auto Mode + ECR repo "sample-app"
terraform/modules/argocd	ArgoCD (Helm chart) + one "root" Application
terragrunt/root.hcl	shared: S3 state bucket + AWS provider
terragrunt/test/env.hcl	environment name + region
terragrunt/test/{network,eks,argocd}/terragrunt.hcl	one "unit" per module
kustomize/base	plain Kubernetes YAML
kustomize/overlays/test	1 replica, namespace sample-app-test
kustomize/overlays/prod	3 replicas, namespace sample-app-prod
helm/sample-app	Helm chart of the same app
argocd/apps	3 ArgoCD Applications: kustomize-test, kustomize-prod, helm

EKS Auto Mode means AWS runs the worker nodes, the load balancer controller and storage for you. There are no node groups to size or patch.

Phase 2 - Edit the values

1. `terragrunt/test/env.hcl`: `name` and `region`.
2. `terragrunt/test/argocd/terragrunt.hcl`: `repo_url` and `repo_branch` must point at **your** repo and branch (ArgoCD reads it).
3. `argocd/apps/*.yaml`: same `repoURL` / `targetRevision`.
4. Replace the account ID `664418953285` in `kustomize/`, `helm/sample-app/values.yaml`.
5. Commit and push. ArgoCD can only deploy what is on GitHub.

Phase 3 - Create the infrastructure (Terragrunt)

```
cd k8s-eks-deploy/terragrunt/test
terragrunt run --all plan          # shows what will be created; nothing is changed yet
terragrunt run --all apply        # type "y" when asked. Takes ~15-20 minutes (EKS is slow to
create)
```

Terragrunt creates the S3 state bucket `tfstate-sample-app-test-<account>-ap-south-1` on first run, then applies **network -> eks -> argocd** in that order, because each unit has a `dependency` block on the one before.

`plan` before the cluster exists uses **mock outputs** (fake VPC/cluster IDs) for the later units. That is normal. The real values are used on `apply`.

Phase 4 - Connect kubectl

```
aws eks update-kubeconfig --region ap-south-1 --name sample-app-test
kubectl get nodes                # may be empty - Auto Mode starts nodes only when pods need them
kubectl get pods -n argocd       # all Running after ~2 minutes
```

Phase 5 - Build and push the image

ECR tags are **immutable** (a tag can never be overwritten), so every build gets a new tag. Start with 1.

```
cd docker-ec2-deploy
ECR=664418953285.dkr.ecr.ap-south-1.amazonaws.com/sample-app
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin ${ECR%/*}
docker build -t $ECR:1 . && docker push $ECR:1
```

Until this image exists, the app pods show `ImagePullBackOff`. That is expected.

Phase 6 - Open ArgoCD

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath='{.data.password}' | base64 -d; echo
kubectl -n argocd port-forward svc/argocd-server 8080:443
```

Open `https://localhost:8080`, user `admin`, the password above. Accept the self-signed certificate warning. You should see 4 apps: `root` and the 3 it created. All should become **Synced / Healthy**.

Phase 7 - Open the app

```
kubectl get svc -A | grep sample-app      # EXTERNAL-IP column = NLB DNS name (takes ~3 minutes)
curl http://<nlb-dns-name>/              # "Hello from sample-app-... (version test)"
```

Phase 8 - Deploy a new version (the GitOps way)

Nobody runs `kubectl apply`. You change Git and ArgoCD does the rest.

```
docker build -t $ECR:2 docker-ec2-deploy && docker push $ECR:2
cd k8s-eks-deploy/kustomize/overlays/test
kustomize edit set image $ECR:2           # rewrites newTag in kustomization.yaml
git commit -am "test: sample-app 2" && git push
```

ArgoCD checks Git every ~3 minutes (or click **Refresh**) and rolls the pods one by one. For Helm, change `image.tag` in `helm/sample-app/values.yaml` (or `valuesObject` in `argocd/apps/sample-app-helm.yaml`) instead.

Try **self-heal**: `kubectl -n sample-app-test scale deploy/sample-app --replicas=5`. ArgoCD puts it back to 1 within seconds, because Git says 1.

Phase 9 - Check your work locally (no cluster needed)

```
kustomize build kustomize/overlays/prod    # final YAML for prod
helm lint helm/sample-app && helm template demo helm/sample-app
```

Troubleshooting

Problem	What to check
<code>terragrunt</code> asks to create the S3 bucket	Answer <code>y</code> . It stores Terraform state; it is created once
Pods <code>ImagePullBackOff</code>	Image tag in Git doesn't exist in ECR yet (Phase 5), or wrong account ID
Pods <code>Pending</code> for minutes	Auto Mode is starting a node; <code>kubectl get nodeclaims</code> shows it. Wait ~2 minutes

Problem	What to check
Service <code>EXTERNAL-IP</code> stays <code><pending></code>	Public subnets are missing the <code>kubernetes.io/role/elb</code> tag (the network module sets it)
ArgoCD app <code>Unknown</code> / repo errors	<code>repoURL</code> / branch wrong, or the repo is private (add it under Settings -> Repositories)
ArgoCD changed back what you edited	That's self-heal. Change Git, not the cluster
<code>kubectl</code> : You must be logged in	Run <code>aws eks update-kubeconfig</code> with the same AWS user that ran Terragrunt
<code>push</code> fails: tag already exists	ECR tags are immutable. Use a new tag number

Cleanup (stop charges)

Delete the apps first, so Kubernetes removes the NLBs before the VPC is deleted:

```
kubectl -n argocd delete application root      # its finalizer removes the 3 apps and their NLBs
kubectl get svc -A | grep LoadBalancer      # wait until none are left
cd k8s-eks-deploy/terragrunt/test
terragrunt run --all destroy                  # argocd -> eks -> network (reverse order)
```

Finally empty and delete the state bucket `tfstate-sample-app-test-<account>-ap-south-1` in the S3 console.

Appendix: project files

All files live in `k8s-eks-deploy/` of the repository.

argocd/apps/sample-app-helm.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: sample-app-helm
  namespace: argocd
  finalizers: [resources-finalizer.argocd.argoproj.io] # deleting the app deletes its resources (and NLB)
spec:
  project: default
  source:
    repoURL: https://github.com/mubinfshaikh/sandbox.git
    targetRevision: docker-ec2-asg-loadbalancer
    path: k8s-eks-deploy/helm/sample-app
    helm:
      valuesObject:
        replicaCount: 1
  destination:
    server: https://kubernetes.default.svc
    namespace: sample-app-helm
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [CreateNamespace=true]
```

argocd/apps/sample-app-kustomize-prod.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: sample-app-kustomize-prod
  namespace: argocd
  finalizers: [resources-finalizer.argocd.argoproj.io] # deleting the app deletes its resources (and NLB)
spec:
  project: default
  source:
    repoURL: https://github.com/mubinfshaikh/sandbox.git
    targetRevision: docker-ec2-asg-loadbalancer
    path: k8s-eks-deploy/kustomize/overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: sample-app-prod
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [CreateNamespace=true]
```

argocd/apps/sample-app-kustomize-test.yaml

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: sample-app-kustomize-test
  namespace: argocd
  finalizers: [resources-finalizer.argocd.argoproj.io] # deleting the app deletes its resources (and NLB)
spec:
  project: default
  source:
    repoURL: https://github.com/mubinfshaikh/sandbox.git
    targetRevision: docker-ec2-asg-loadbalancer
    path: k8s-eks-deploy/kustomize/overlays/test
  destination:
    server: https://kubernetes.default.svc
    namespace: sample-app-test
  syncPolicy:
    automated: { prune: true, selfHeal: true }
    syncOptions: [CreateNamespace=true]
```

helm/sample-app/Chart.yaml

```
apiVersion: v2
name: sample-app
description: Sample Node app (same app as docker-ec2-deploy/app)
type: application
version: 0.1.0
appVersion: "1"
```

helm/sample-app/templates/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}
  labels: { app: {{ .Release.Name }} }
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels: { app: {{ .Release.Name }} }
  template:
    metadata:
      labels: { app: {{ .Release.Name }} }
    spec:
      containers:
        - name: app
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports: [{ containerPort: 3000 }]
          env:
            - name: APP_VERSION
              value: {{ .Values.image.tag | quote }}
          readinessProbe:
            httpGet: { path: /health, port: 3000 }
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /health, port: 3000 }
            periodSeconds: 10
          resources: {{- toYaml .Values.resources | nindent 12 }}
```

helm/sample-app/templates/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
spec:
  type: LoadBalancer
  loadBalancerClass: eks.amazonaws.com/nlb
  selector: { app: {{ .Release.Name }} }
  ports: [{ port: {{ .Values.service.port }}, targetPort: 3000 }]
```

helm/sample-app/values.yaml

```
replicaCount: 2
image:
  repository: 664418953285.dkr.ecr.ap-south-1.amazonaws.com/sample-app
  tag: "1"
service:
  port: 80
resources:
  requests: { cpu: 50m, memory: 64Mi }
  limits: { memory: 128Mi }
```

kustomize/base/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sample-app
spec:
  replicas: 2
  selector:
    matchLabels: { app: sample-app }
  template:
    metadata:
      labels: { app: sample-app }
    spec:
      containers:
        - name: app
          image: 664418953285.dkr.ecr.ap-south-1.amazonaws.com/sample-app:latest
          ports: [{ containerPort: 3000 }]
          env:
            - name: APP_VERSION
              value: base
          readinessProbe:
            httpGet: { path: /health, port: 3000 }
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /health, port: 3000 }
            periodSeconds: 10
          resources:
            requests: { cpu: 50m, memory: 64Mi }
            limits: { memory: 128Mi }
```

kustomize/base/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources: [deployment.yaml, service.yaml]
```

kustomize/base/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: sample-app
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
spec:
  type: LoadBalancer
  loadBalancerClass: eks.amazonaws.com/nlb # EKS Auto Mode creates an NLB
  selector: { app: sample-app }
  ports: [{ port: 80, targetPort: 3000 }]
```

kustomize/overlays/prod/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: sample-app-prod
resources: [../base]
images:
  - name: 664418953285.dkr.ecr.ap-south-1.amazonaws.com/sample-app
    newTag: "1" # CI bumps this: kustomize edit set image 664418953285.dkr.ecr.ap-south-1.
amazonaws.com/sample-app:<tag>
replicas:
  - name: sample-app
    count: 3
patches:
  - target: { kind: Deployment, name: sample-app }
    patch: |-
      - op: replace
        path: /spec/template/spec/containers/0/env/0/value
        value: prod
```

kustomize/overlays/test/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: sample-app-test
resources: [../../base]
images:
  - name: 664418953285.dkr.ecr.ap-south-1.amazonaws.com/sample-app
    newTag: "1" # CI bumps this: kustomize edit set image 664418953285.dkr.ecr.ap-south-1.
amazonaws.com/sample-app:<tag>
replicas:
  - name: sample-app
    count: 1
patches:
  - target: { kind: Deployment, name: sample-app }
    patch: |-
      - op: replace
        path: /spec/template/spec/containers/0/env/0/value
        value: test
```

terraform/modules/argocd/main.tf

```
resource "helm_release" "argocd" {
  name           = "argocd"
  repository     = "https://argoproj.github.io/argo-helm"
  chart          = "argo-cd"
  version        = var.argocd_chart_version
  namespace      = "argocd"
  create_namespace = true
}

# App-of-apps: one root Application that syncs every Application in argocd/apps/
resource "helm_release" "root_app" {
  name           = "argocd-root"
  repository     = "https://argoproj.github.io/argo-helm"
  chart          = "argocd-apps"
  version        = var.argocd_apps_chart_version
  namespace      = "argocd"

  values = [yamlencode({
    applications = {
      root = {
        project      = "default"
        finalizers   = ["resources-finalizer.argocd.argoproj.io"]
        source = {
          repoURL      = var.repo_url
          targetRevision = var.repo_branch
          path          = "k8s-eks-deploy/argocd/apps"
        }
      }
      destination = { server = "https://kubernetes.default.svc", namespace = "argocd" }
      syncPolicy  = { automated = { prune = true, selfHeal = true } }
    }
  })]

  depends_on = [helm_release.argocd]
}
```

terraform/modules/argocd/variables.tf

```
variable "repo_url" { type = string }
variable "repo_branch" { type = string }
variable "argocd_chart_version" { type = string }
variable "argocd_apps_chart_version" { type = string }
```

terraform/modules/eks/main.tf

```
module "eks" {
  source = "terraform-aws-modules/eks/aws"
  version = "~> 21.26"

  name = var.name
  kubernetes_version = var.kubernetes_version

  vpc_id = var.vpc_id
  subnet_ids = var.subnet_ids

  endpoint_public_access = true
  enable_cluster_creator_admin_permissions = true

  # EKS Auto Mode: AWS manages nodes, load balancers and storage - no node groups to maintain
  compute_config = {
    enabled = true
    node_pools = ["general-purpose"]
  }
}

resource "aws_ecr_repository" "app" {
  name = var.ecr_repo
  image_tag_mutability = "IMMUTABLE"
  force_delete = true
  image_scanning_configuration { scan_on_push = true }
}
```

terraform/modules/eks/outputs.tf

```
output "cluster_name" { value = module.eks.cluster_name }
output "cluster_endpoint" { value = module.eks.cluster_endpoint }
output "cluster_certificate_authority_data" { value = module.eks.cluster_certificate_authority_data }
output "ecr_repository_url" { value = aws_ecr_repository.app.repository_url }
```

terraform/modules/eks/variables.tf

```
variable "name" { type = string }
variable "kubernetes_version" { type = string }
variable "vpc_id" { type = string }
variable "subnet_ids" { type = list(string) }
variable "ecr_repo" { type = string }
```

terraform/modules/network/main.tf

```
data "aws_availability_zones" "available" { state = "available" }

locals { azs = slice(data.aws_availability_zones.available.names, 0, 2) }

module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "~> 6.7"

  name          = var.name
  cidr          = var.cidr
  azs           = local.azs
  public_subnets = [for i, _ in local.azs : cidrsubnet(var.cidr, 4, i)]
  private_subnets = [for i, _ in local.azs : cidrsubnet(var.cidr, 4, i + 8)]

  # ponytail: one NAT for the whole VPC (~$35/mo); one per AZ if an AZ outage must not stop
  egress
  enable_nat_gateway = true
  single_nat_gateway = true

  # tags EKS uses to place internet-facing / internal load balancers
  public_subnet_tags = { "kubernetes.io/role/elb" = 1 }
  private_subnet_tags = { "kubernetes.io/role/internal-elb" = 1 }
}
```

terraform/modules/network/outputs.tf

```
output "vpc_id" { value = module.vpc.vpc_id }
output "private_subnets" { value = module.vpc.private_subnets }
```

terraform/modules/network/variables.tf

```
variable "name" { type = string }
variable "cidr" {
  type    = string
  default = "10.0.0.0/16"
}
```

terragrunt/root.hcl

```
# Shared by every unit: S3 state per unit + AWS provider.
locals {
  env = read_terragrunt_config(find_in_parent_folders("env.hcl")).locals
}

remote_state {
  backend = "s3"
  generate = { path = "backend.tf", if_exists = "overwrite_terragrunt" }
  config = {
    bucket      = "tfstate-${local.env.name}-${get_aws_account_id()}-${local.env.region}"
    key         = "${path_relative_to_include()}/terraform.tfstate"
    region      = local.env.region
    encrypt     = true
    use_lockfile = true
  }
}

generate "provider" {
  path      = "provider.tf"
  if_exists = "overwrite_terragrunt"
  contents = <<-EOT
    terraform {
      required_version = ">= 1.10"
      required_providers {
        aws = { source = "hashicorp/aws", version = "~> 6.67" }
        helm = { source = "hashicorp/helm", version = "~> 3.3" }
      }
    }
    provider "aws" {
      region = "${local.env.region}"
      default_tags { tags = { Project = "${local.env.name}", ManagedBy = "terragrunt" } }
    }
  EOT
}
```

terragrunt/test/argocd/terragrunt.hcl

```
include "root" { path = find_in_parent_folders("root.hcl") }

locals { env = read_terragrunt_config(find_in_parent_folders("env.hcl")).locals }

terraform { source = "${get_repo_root()}/k8s-eks-deploy/terraform/modules/argocd" }

dependency "eks" {
  config_path = "../eks"
  mock_outputs = {
    cluster_name           = "mock"
    cluster_endpoint       = "https://mock.example.com"
    cluster_certificate_authority_data = "bw9jaw=="
  }
  mock_outputs_allowed_terraform_commands = ["init", "validate", "plan"]
}

# The helm provider needs the cluster, so it is generated here rather than in root.hcl.
generate "helm_provider" {
  path      = "helm_provider.tf"
  if_exists = "overwrite_terragrunt"
  contents = <<-EOT
    provider "helm" {
      kubernetes = {
        host = "${dependency.eks.outputs.cluster_endpoint}"
        cluster_ca_certificate = base64decode("${dependency.eks.outputs.cluster_certificate_
authority_data}")
        exec = {
          api_version = "client.authentication.k8s.io/v1beta1"
          command      = "aws"
          args          = ["eks", "get-token", "--region", "${local.env.region}", "--cluster-n
ame", "${dependency.eks.outputs.cluster_name}"]
        }
      }
    }
  EOT
}

inputs = {
  repo_url           = "https://github.com/mubinfshaikh/sandbox.git"
  repo_branch        = "docker-ec2-asg-loadbalancer"
  argocd_chart_version = "10.9.6"
  argocd_apps_chart_version = "2.0.6"
}
```

terragrunt/test/eks/terragrunt.hcl

```
include "root" { path = find_in_parent_folders("root.hcl") }

locals { env = read_terragrunt_config(find_in_parent_folders("env.hcl")).locals }

terraform { source = "${get_repo_root()}/k8s-eks-deploy/terraform/modules/eks" }

dependency "network" {
  config_path          = "../network"
  mock_outputs         = { vpc_id = "vpc-mock", private_subnets = ["subne
t-mock1", "subnet-mock2"] }
  mock_outputs_allowed_terraform_commands = ["init", "validate", "plan"]
}

inputs = {
  name              = local.env.name
  kubernetes_version = "1.37"
  vpc_id            = dependency.network.outputs.vpc_id
  subnet_ids        = dependency.network.outputs.private_subnets
  ecr_repo           = "sample-app"
}
```

terragrunt/test/env.hcl

```
locals {
  name  = "sample-app-test"
  region = "ap-south-1"
}
```

terragrunt/test/network/terragrunt.hcl

```
include "root" { path = find_in_parent_folders("root.hcl") }

locals { env = read_terragrunt_config(find_in_parent_folders("env.hcl")).locals }

terraform { source = "${get_repo_root()}/k8s-eks-deploy/terraform/modules/network" }

inputs = {
  name = local.env.name
  cidr = "10.20.0.0/16"
}
```